# Case Study 8: Student Marks Matrix Analysis

Activity sheet case study #42 | Concepts: Nested lists, nested loops, lists, strings, conditional statements, algorithms (maximum search)

## 1. Problem Statement

Marks of several students in several subjects are stored in a matrix (nested list). Compute row-wise statistics (total and average per student), column-wise statistics (average, highest, lowest and failures per subject), identify the top performer(s) and detect subjects whose class average is low.

## 2. Objectives

- Represent tabular data as a nested list and traverse it with nested loops.
- Compute row-wise and column-wise statistics.
- Find maximum values together with the owner of the value, including ties.
- Validate the shape of the matrix before processing.

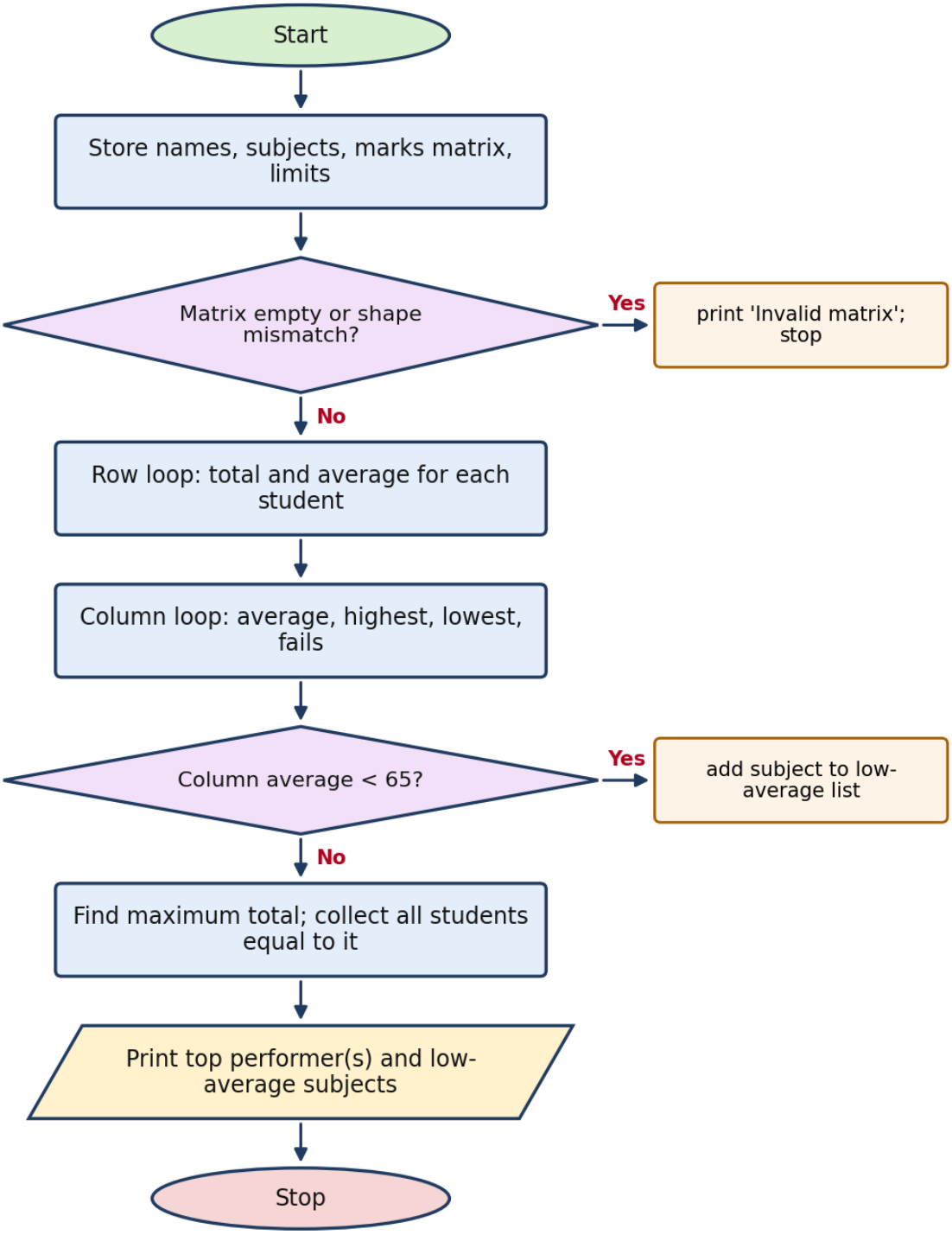
## 3. Assumptions

- marks[i][j] is the mark of student i in subject j, out of 100.
- Pass mark is 40; a subject has a low average when its mean is below 65.
- The matrix is valid only if it has at least one student, one row per student name and one column per subject.
- All students with the highest total are reported as top performers.

## 4. Algorithm

1. Store names, subjects and the marks matrix; set PASS_MARK and LOW_AVERAGE.
2. Validate: row count equals number of names, every row length equals number of subjects, and the matrix is not empty.
3. For every row, add its marks to obtain the total and the average; print the row and save the total.
4. For every column, scan all rows to find the sum, highest (with the student), lowest and the count of marks below the pass mark.
5. Print the column statistics; if the average is below the low-average limit, mark the subject and add it to the low list.
6. Find the maximum total, then collect every student whose total equals it.
7. Print the top performer(s) and the list of low-average subjects (or a message if none).

## 5. Flowchart




## 6. Pseudocode

```
START
  READ names, subjects, marks (nested list)
  IF rows = 0 THEN PRINT "No student data" ; STOP
  valid <- (rows = LENGTH(names))
  FOR each row in marks
      IF LENGTH(row) != LENGTH(subjects) THEN valid <- false
  IF NOT valid THEN PRINT "Invalid matrix" ; STOP
  FOR i = 0 TO rows-1                       (row-wise statistics)
      total <- 0
      FOR j = 0 TO cols-1 : total <- total + marks[i][j]
```

```
      totals[i] <- total ; PRINT row, total, total/cols
  FOR j = 0 TO cols-1                     (column-wise statistics)
      sum, highest, lowest, fails <- scan marks[i][j] for all i
      average <- sum / rows
      IF average < LOW_AVERAGE THEN add subject to low list
      PRINT subject statistics
  best <- maximum of totals
  top <- names whose total = best
  PRINT top and low-average subjects
STOP
```

## 7. Python Source Code

```
# Case Study 42: Student Marks Matrix Analysis
# ---- INPUT DATA ----
names = ["Anil", "Bina", "Chetan", "Divya", "Eshan", "Farah"]
subjects = ["Maths", "Physics", "Chemistry", "English"]
marks = [[78, 65, 70, 82],
         [92, 88, 95, 90],
         [45, 38, 52, 60],
         [67, 72, 58, 75],
         [55, 40, 48, 66],
         [88, 79, 84, 91]]
# ---- END DATA ----

PASS_MARK = 40
LOW_AVERAGE = 65

rows = len(marks)
cols = len(subjects)

valid = True
if rows != len(names):
    valid = False
for row in marks:
    if len(row) != cols:
        valid = False

if rows == 0 and len(names) == 0:
    print("No student data to analyse.")
elif not valid or rows == 0:
    print("Invalid matrix: rows/columns do not match the names and subjects.")
else:
    # ---------- row-wise statistics ----------
    header = "Name".ljust(9)
    for s in subjects:
        header = header + s.rjust(11)
    header = header + "Total".rjust(8) + "Average".rjust(9)
    print(header)
    print("-" * len(header))
    totals = []
    for i in range(rows):
        total = 0
        line = names[i].ljust(9)
        for j in range(cols):
            total = total + marks[i][j]
            line = line + str(marks[i][j]).rjust(11)
        totals.append(total)
        line = line + str(total).rjust(8) + "{:.2f}".format(total / cols).rjust(9)
        print(line)

    # ---------- column-wise statistics ----------
    print()
    print("{:<11}{:>9}{:>16}{:>16}{:>7}   {}".format(
          "Subject", "Average", "Highest", "Lowest", "Fails", "Remark"))
    low_subjects = []
    for j in range(cols):
        col_sum = 0
        highest = marks[0][j]
        highest_name = names[0]
        lowest = marks[0][j]
        lowest_name = names[0]
        fails = 0
        for i in range(rows):
            value = marks[i][j]
            col_sum = col_sum + value
            if value > highest:
                highest = value
                highest_name = names[i]
            if value < lowest:
                lowest = value
                lowest_name = names[i]
            if value < PASS_MARK:
                fails = fails + 1
        average = col_sum / rows
        remark = ""
        if average < LOW_AVERAGE:
            remark = "LOW AVERAGE"
            low_subjects.append(subjects[j])
        print("{:<11}{:>9.2f}{:>16}{:>16}{:>7}   {}".format(
              subjects[j], average,
              str(highest) + " (" + highest_name + ")",
```

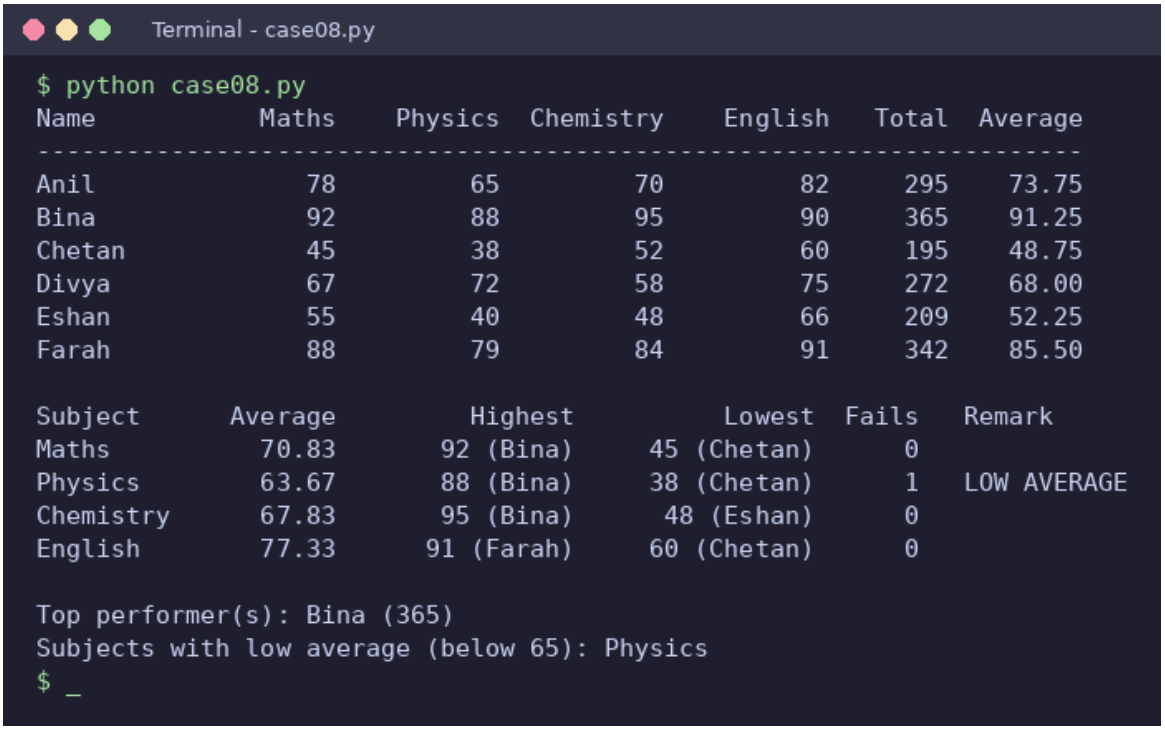
```
80                str(lowest) + " (" + lowest_name + ")",
81                fails, remark))
82
83      # ---------- top performers ----------
84      best = totals[0]
85      for t in totals:
86          if t > best:
87              best = t
88      top_names = []
89      for i in range(rows):
90          if totals[i] == best:
91              top_names.append(names[i])
92      print()
93      print("Top performer(s): " + ", ".join(top_names) + " (" + str(best) + ")")
94      if len(low_subjects) == 0:
95          print("No subject has a low average.")
96      else:
97          print("Subjects with low average (below " + str(LOW_AVERAGE) + "): "
98                + ", ".join(low_subjects))
```

## 8. Test Cases

| # | Test case | Input / data | Expected result | Observed output (actual) | Status |
|---|---|---|---|---|---|
| 1 | Default 6 x 4 matrix | 6 students, 4 subjects | Bina tops; Physics flagged | `Top performer(s): Bina (365) \| Subjects with low average (below 65): Physics` | **PASS** |
| 2 | Tie for top total | two students, totals 160 and 160 | Both students reported | `Top performer(s): A, B (160)` | **PASS** |
| 3 | No low-average subject | one student [70, 80] | Single-row matrix works | `Top performer(s): Solo (150) \| No subject has a low average.` | **PASS** |
| 4 | Names and rows differ | 2 names, 1 row | Shape check | `Invalid matrix: rows/columns do not match the names and subjects.` | **PASS** |
| 5 | Ragged row | second row has 1 mark | Shape check | `Invalid matrix: rows/columns do not match the names and subjects.` | **PASS** |
| 6 | Empty data | no students | No crash | `No student data to analyse.` | **PASS** |
| 7 | Failing marks counted | a mark of 35 in S1 | Average 62.5 flagged; one fail counted | `S1 62.50 90 (B) 35 (A) 1 LOW AVERAGE` | **PASS** |

## 9. Output Screenshot

Terminal output of `python case08.py` using the default data:



## 10. Conclusion

Nested loops make it straightforward to scan a matrix by rows and by columns. The program computes all required statistics, handles ties for the top position, flags weak subjects and refuses malformed matrices instead of producing wrong results.